PROJECT ABSTRACT

NetBridge Offline Payment Research Prototype

Abstract and approach

Abstract

NetBridge is a simulation-only Progressive Web App (PWA) that explores how payment requests can be authorized and recorded when network access is intermittent. In its face-to-face flow, a merchant dashboard creates a request QR. A customer scans it in the mobile web app, reviews the amount, approves with a demo PIN and Authenticator code, and displays a 10-character confirmation code for the merchant to enter. Once both app pages have been cached, the handoff can be demonstrated without Wi-Fi; each device retains its own local record. A separate far-recipient simulation lets a customer enter a NetBridge ID and amount. An offline send stays pending on the sender device and can reach the recipient only after the sender reconnects to the NetBridge server. The prototype uses Flask, SQLite, browser JavaScript, QR libraries, a Service Worker, and local browser storage. It demonstrates interface and workflow choices, not financial settlement or production-grade fraud prevention.

Keywords: offline-first PWA, simulated payments, QR request, Authenticator, deferred synchronization

1. Problem and objective

The project studies two different connectivity constraints: a nearby customer and merchant need a clear local handoff, while a far recipient cannot receive a server-recorded payment until the sending device reconnects. The objective is to make both paths understandable and testable in a browser prototype without implying that a real payment has settled.

2. Scope

- Face-to-face mode: merchant request QR, customer review and authorization, then a short code matched against the active request.
- Far-recipient mode: recipient NetBridge ID and amount, local pending state while offline, then server synchronization when the sender reconnects.
- PWA behavior: cached app shell and local demo wallet/history support after initial setup.
- Excluded: real UPI or bank integration, money movement, production identity proof, and global double-spend protection across disconnected devices.

3. Approach and system design

The implementation separates client-side demonstrations from server-dependent operations. The customer PWA and merchant dashboard are browser interfaces. Flask provides setup and API routes, with SQLite storing fictional server-side demo state. The Service Worker caches the client shell and static libraries; local browser storage supports the offline user experience. Far-recipient delivery uses the server after reconnection.

3.1 Face-to-face offline handoff

- Merchant enters an amount and demo PIN to create a time-limited payment request QR.
- Customer scans the request, checks the merchant and amount, then approves with the demo PIN and Authenticator code.
- Customer displays a 10-character mixed-case confirmation code. The merchant types the code and checks it against the active request.
- Merchant records a simulation receipt locally. Without a return communication channel, the phone cannot know that the merchant accepted it.

3.2 Far-recipient simulation

- Sender enters a recipient NetBridge ID and amount; this flow does not use QR scanning or the customer confirmation code.
- If offline, the sender keeps the request pending on that device.
- After connectivity returns, the sender can synchronize with Flask; the recipient sees the simulated delivery through the server-backed view.
- This is deferred demo synchronization, not offline settlement between phones. The receiver needs the server to receive the transaction.

3.3 Components

- Customer PWA: responsive web UI, QR camera reader, Authenticator screens, Service Worker, and local demo state.
- Merchant dashboard: request QR generation, typed-code entry, merchant demo PIN, and transaction history.
- Flask and SQLite: enrollment and demo API routes, server-side request handling, and far-payment synchronization.
- QR payloads and confirmation codes support the prototype workflow. They are not cryptographic proof of identity or a bank transfer.

4. Interface examples

These are representative static renders based on the current customer and merchant page structure and styles. The displayed values and activity are illustrative sample data, not live account state.

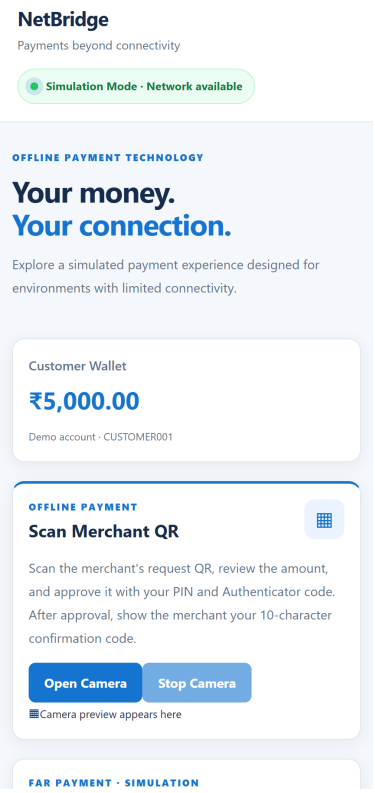


Figure 1. Customer mobile view

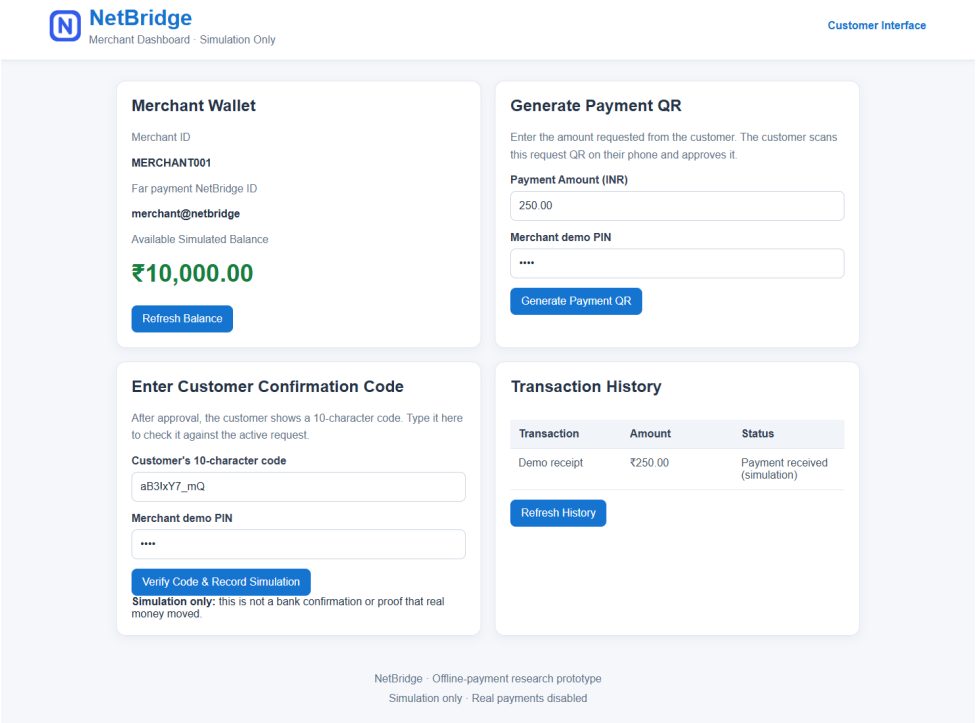


Figure 2. Merchant dashboard

5. Security, limitations, and evaluation

The demo Authenticator check, local storage, and short confirmation code support a classroom workflow exercise only. Client-side checks and cached state can be altered on a compromised device; separate offline copies cannot coordinate to prevent global double spending. The far-recipient flow depends on server reachability at synchronization time. No financial network or settlement service is connected.

Evaluation should exercise accepted and rejected codes, expiry and replay handling, offline face-to-face completion after caching, persistence across reloads, and far-payment behavior before and after server reconnection. Usability review should check that each screen makes the simulated status and next action clear. These are proposed checks; this document does not report a new test run.

Presentation language: Say "authorized or recorded in the simulation." Do not say that money transferred or settled.